SOFTWARE REQUIREMENTS SPECIFICATION

for

HOSPITAL MANAGEMENT SYSTEM

SE MINI PROJECT

C / C++

Sl. No.	SRN	Name
1	PES2UG24CS369	PRATHAM P MUDAKAVI
2	PES2UG24CS370	Pratham P Shetty
3	PES2UG24CS371	PRATHIK M NAMBIAR
4	PES2UG24CS372	PRATIK PATIL

Team 1

Prepared for Software Engineering Mini Project

Table of Contents

- 1. Introduction
- 2. Overall Description
- 3. Functional Requirements
- 4. Non-Functional Requirements
- 5. External Interface Requirements
- 6. System Models and Data Design
- 7. Detailed Module Specifications
- 8. System Requirements and Constraints
- 9. Use Cases
- 10. Testing and Acceptance Criteria
- 11. Future Enhancements
- 12. Conclusion
- Appendix A – Data Dictionary

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to define the functional and non-functional requirements of the Hospital Management System (HMS). The system is proposed as a C/C++ based mini project for managing routine hospital information and operations in a structured and efficient manner. This document establishes a common understanding between the project team, evaluator, and intended users regarding what the system should do and the constraints under which it will operate.

1.2 Scope

The Hospital Management System will provide a centralized computerized mechanism for maintaining patient, doctor, appointment, ward/room, medical record, and billing information. It will support data entry, searching, updating, deletion where permitted, and generation of basic reports. The mini project is intended primarily for academic demonstration and can be implemented as a console-based or simple desktop application using C or C++.

- Register new patients and maintain unique patient records.
- Store and manage doctor and staff information.
- Schedule, view, modify, and cancel appointments.
- Maintain admission, ward, room, and discharge information.
- Record diagnoses, treatments, prescriptions, and related medical information.
- Generate and maintain patient billing information.
- Search records using identifiers such as patient ID, doctor ID, or appointment ID.
- Provide summary reports useful to hospital administration.

1.3 Intended Audience

- Hospital receptionist or front-desk operator
- Doctors and authorized medical staff
- Billing/administrative staff
- Hospital administrator
- Project evaluators, faculty, and developers

1.4 Definitions, Acronyms, and Abbreviations

Term	Meaning
HMS	Hospital Management System
SRS	Software Requirements Specification
Patient ID	Unique identifier assigned to a patient
Doctor ID	Unique identifier assigned to a doctor
Appointment ID	Unique identifier for an appointment
CRUD	Create, Read, Update, Delete
UI	User Interface
DB/File Storage	Persistent mechanism used to store system records

1.5 References

This SRS follows commonly used Software Engineering requirements-documentation practices. The implementation is expected to use standard C/C++ language features and file or structured data storage appropriate for a mini project.

2. Overall Description

2.1 Product Perspective

The HMS is a standalone application intended to replace fragmented manual record keeping with a structured information-management solution. The system can initially operate on a single computer and use local persistent storage. Its modular design should permit future migration to a database-backed or networked architecture.

2.2 Product Functions

1. Patient registration and demographic information management.
2. Doctor and department management.
3. Appointment creation and scheduling.
4. Patient admission, room allocation, and discharge.
5. Medical history and treatment record management.
6. Prescription and medicine information recording.
7. Billing and payment record management.
8. Search, filtering, and retrieval of records.
9. Administrative reports and summaries.

2.3 User Classes and Characteristics

User Class	Responsibilities	Expected Skill
Administrator	Manage users, doctors, departments, rooms, and reports.	Basic computer skills
Receptionist	Register patients and manage appointments.	Basic computer skills
Doctor	View patient records and enter diagnosis/treatment information.	Basic system training
Billing Staff	Create bills and record payments.	Basic computer skills

2.4 Operating Environment

- Operating System: Windows or Linux.
- Programming Language: C or C++.
- Compiler: GCC/G++, MinGW, or an equivalent C/C++ compiler.
- Storage: Local text, CSV, binary, or other suitable persistent files.
- Hardware: Standard personal computer with keyboard, display, and sufficient storage.

2.5 Design and Implementation Constraints

- The mini project shall be implemented using C/C++.
- The initial version is expected to be suitable for a small hospital or academic demonstration.
- User access should be restricted according to role where authentication is implemented.
- Input validation is required to reduce incorrect or inconsistent records.
- The design should avoid duplicate identifiers for core entities.

2.6 Assumptions and Dependencies

- Users have basic computer literacy and receive minimal training.
- Hospital staff provide valid patient and operational information.

- The computer has sufficient local storage for project-scale records.
- The system does not directly interface with external insurance, laboratory, or payment gateways in the initial release.
- The project may use file handling instead of a full database to satisfy the C/C++ mini-project scope.

3. Functional Requirements

Functional requirements describe the services that the Hospital Management System must provide. Each requirement is assigned an identifier and priority for traceability.

ID	Requirement	Description	Priority
FR-01	User Login and Access	The system shall authenticate authorized users and provide access to functions appropriate to their role.	High

ID	Requirement	Description	Priority
FR-02	Patient Registration	The system shall create a unique patient record containing patient ID, name, age/date of birth, gender, contact information, address, and emergency contact.	High

ID	Requirement	Description	Priority
FR-03	Patient Record Update	The system shall allow authorized users to update permitted patient details while preserving the patient ID.	High

ID	Requirement	Description	Priority
FR-04	Patient Search	The system shall search for patients by patient ID, name, contact number, or other supported fields.	High

ID	Requirement	Description	Priority
FR-05	Doctor Management	The system shall store doctor ID, name, specialization, department, contact information, and availability.	High

ID	Requirement	Description	Priority
FR-06	Appointment Scheduling	The system shall create appointments linking a patient with a doctor, date, time, and purpose.	High

ID	Requirement	Description	Priority
FR-07	Appointment Modification	The system shall allow authorized users to reschedule or cancel appointments and record the updated status.	High

ID	Requirement	Description	Priority
FR-08	Admission Management	The system shall record admission details including admission date, ward/room, attending doctor, and reason for admission.	High

ID	Requirement	Description	Priority
FR-09	Room Allocation	The system shall display available rooms/beds and assign an available unit to an admitted patient.	Medium

ID	Requirement	Description	Priority
FR-10	Medical Record Management	The system shall store diagnosis, symptoms, treatment notes, prescriptions, and relevant visit information.	High

ID	Requirement	Description	Priority
FR-11	Discharge Management	The system shall record discharge date, discharge summary, and final status for admitted patients.	Medium

ID	Requirement	Description	Priority
FR-12	Billing	The system shall create bills based on applicable consultation, room, procedure, medicine, and other charges entered into the system.	High

ID	Requirement	Description	Priority
FR-13	Payment Recording	The system shall record payment amount, payment date, payment status, and outstanding balance.	High

ID	Requirement	Description	Priority
FR-14	Reports	The system shall generate basic reports such as patient lists, appointments, admissions, room occupancy, and billing summaries.	Medium

ID	Requirement	Description	Priority
FR-15	Data Validation	The system shall validate mandatory fields, identifiers, dates, numeric values, and duplicate records before saving data.	High

ID	Requirement	Description	Priority
FR-16	Data Persistence	The system shall save records to persistent storage and load them when the application starts.	High

ID	Requirement	Description	Priority
FR-17	Data Backup	The system should provide a simple mechanism for copying or exporting important data files.	Medium

ID	Requirement	Description	Priority
FR-18	Exit and Safe Save	The system shall close gracefully and ensure that committed records are not lost.	Medium

4. Non-Functional Requirements

4.1 Performance

- For project-scale datasets, common searches should normally complete within a few seconds.
- The system should respond promptly to menu selections and data-entry operations.
- File operations should be performed efficiently enough for the expected academic dataset.

4.2 Usability

- Menus and prompts should be clear and consistent.
- The application should display meaningful error messages for invalid input.
- Frequently used operations should require a reasonable number of steps.
- The system should show confirmation after successful create/update/delete operations.

4.3 Reliability

- The system should prevent malformed records from being stored.
- Unexpected invalid input should not terminate the application.
- Persistent files should be handled safely to minimize data corruption.
- The application should maintain consistent relationships between patient, doctor, appointment, admission, and billing records.

4.4 Security

- Only authorized users should access protected administrative functions.
- Sensitive patient information should not be unnecessarily displayed to unauthorized users.
- Administrative operations should be separated from ordinary receptionist operations where role-based access is implemented.
- Passwords, if stored, should not be displayed in plain text on the user interface.

4.5 Maintainability

- The program should be divided into logical modules/functions.
- Structures/classes should represent major entities such as Patient, Doctor, Appointment, and Bill.
- Constants and validation rules should be centralized where practical.
- Functions should have clear names and limited responsibilities.

4.6 Portability

- The source code should compile on commonly available C/C++ compilers with minimal changes.
- The system should avoid unnecessary platform-specific dependencies.

5. External Interface Requirements

5.1 User Interface

The initial implementation may use a console-based menu interface. A typical main menu can include:

10. Login / User Authentication
11. Patient Management
12. Doctor Management
13. Appointment Management
14. Admission and Room Management
15. Medical Records
16. Billing and Payments
17. Reports
18. Backup / Data Management
19. Exit

5.2 Hardware Interface

- Keyboard for data entry.
- Monitor/display for menus, forms, records, and reports.
- Local storage for program and persistent data files.

5.3 Software Interface

- C/C++ compiler and standard library.
- Operating-system file system for persistent storage.
- Optional development environment such as Visual Studio Code, Code::Blocks, Visual Studio, or another C/C++ IDE.

5.4 Communication Interface

No external network communication is required for the initial version. The architecture should however keep modules sufficiently independent to allow future integration with network services, email/SMS notifications, or external databases.

6. System Models and Data Design

6.1 High-Level System Flow

User → Login → Main Menu → Select Module → Validate Input → Process Operation → Read/Write Persistent Data → Display Result → Return to Main Menu.

6.2 Use Case Model

Actor	Major Use Cases
Administrator	Login, manage users, doctors, departments, rooms, reports, backup
Receptionist	Register patient, search patient, schedule appointment, admission/discharge support
Doctor	Login, view patient record, add diagnosis, treatment, prescription
Billing Staff	Create bill, record payment, view outstanding balance, billing reports

6.3 Entity Relationship Description

The core data model consists of Patient, Doctor, Appointment, Admission, Room, MedicalRecord, Prescription, Bill, and Payment entities. A patient may have multiple appointments and medical records. A doctor may have multiple appointments and medical records. An admission links a patient to an attending doctor and an allocated room. A bill is associated with a patient and may have one or more payment entries.

Entity	Primary Key	Important Attributes
Patient	PatientID	Name, DOB/Age, Gender, Contact, Address, EmergencyContact
Doctor	DoctorID	Name, Specialization, Department, Contact, Availability
Appointment	AppointmentID	PatientID, DoctorID, Date, Time, Purpose, Status
Admission	AdmissionID	PatientID, DoctorID, RoomID, AdmissionDate, DischargeDate,

		Status
Room	RoomID	Ward, RoomNumber, BedNumber, Type, Availability
MedicalRecord	RecordID	PatientID, DoctorID, Date, Diagnosis, Treatment, Notes
Prescription	PrescriptionID	RecordID, Medicine, Dosage, Frequency, Duration
Bill	BillID	PatientID, Date, Charges, Total, Paid, Balance
Payment	PaymentID	BillID, Date, Amount, Method, Status

6.4 Data Flow Description

- Patient data flows from registration input to validation and persistent patient storage.
- Appointment data references existing patient and doctor IDs before being saved.
- Medical records reference the patient and attending doctor.
- Admission data references the patient and an available room.
- Billing data references the patient and accumulates applicable charges.
- Reports retrieve validated records and present summarized information to authorized users.

7. Detailed Module Specifications

7.1 Patient Management Module

- Add a new patient with a generated/validated Patient ID.
- Display a patient's complete basic profile.
- Update permitted demographic/contact details.
- Search by ID or name.
- Prevent duplicate Patient IDs.
- Maintain patient status such as active, admitted, or discharged.

7.2 Doctor Management Module

- Add and update doctor information.
- Store specialization and department.
- Display doctor availability.
- Search doctors by ID, name, specialization, or department.

7.3 Appointment Module

- Create appointments only for existing patients and doctors.
- Validate date/time input.
- Maintain appointment status: Scheduled, Completed, Cancelled, or Rescheduled.
- Display appointments for a patient or doctor.
- Prevent obvious duplicate/conflicting bookings where availability data permits.

7.4 Admission and Room Module

- Create an admission record for a patient.
- Display available rooms/beds.

- Assign an available room or bed.
- Update room occupancy when a patient is admitted or discharged.
- Store discharge date and discharge status.

7.5 Medical Records Module

- Create a clinical record for a patient visit/admission.
- Store diagnosis, treatment notes, and prescriptions.
- Allow authorized medical staff to retrieve historical records.
- Link records to the responsible doctor.

7.6 Billing Module

- Create a bill for a patient.
- Record individual charge categories.
- Calculate subtotal/total and outstanding balance.
- Record payments and update payment status.
- Generate a printable/displayable billing summary.

7.7 Reporting Module

- Generate patient lists.
- Generate appointment schedules.
- Generate admission/occupancy summaries.
- Generate billing and outstanding-payment summaries.
- Provide counts useful for basic administration.

8. System Requirements and Constraints

8.1 Minimum Hardware Requirements

Component	Minimum Requirement
Processor	Any modern dual-core processor or equivalent
RAM	4 GB or more
Storage	At least 500 MB free for application and project data
Display	Standard monitor with keyboard

8.2 Software Requirements

Software	Requirement
Operating System	Windows 10/11 or a compatible Linux distribution
Language	C or C++
Compiler	GCC/G++ or equivalent
IDE	Any C/C++ compatible IDE/editor
Storage	Text/CSV/binary files or equivalent local persistence

8.3 Constraints

- The project is limited to the scope appropriate for a Software Engineering mini project.
- The first release does not require integration with real hospital devices or external healthcare systems.
- The first release does not require online appointment booking or cloud deployment.

- The system is not intended to replace professional medical judgment.
- Access to sensitive information must be controlled in accordance with the intended academic demonstration environment.

9. Use Cases

ID	Use Case	Primary Actor	Precondition	Main Flow	Exception/Alternative
UC-01	Register Patient	Receptionist	Patient does not already exist	1. Select Patient Management. 2. Select Register Patient. 3. Enter required details. 4. Validate input. 5. Generate/confirm Patient ID. 6. Save record. 7. Display confirmation.	Invalid input → show error and request correction.
UC-02	Book Appointment	Receptionist	Patient and doctor exist	1. Select Appointment Management. 2. Enter Patient ID. 3. Select Doctor. 4. Enter date/time. 5. Check availability. 6. Save appointment. 7. Display appointment ID.	Conflict or invalid ID → reject and request correction.
UC-03	Update Medical Record	Doctor	Authorized doctor is logged in and patient exists	1. Search patient. 2. Open medical record. 3. Enter diagnosis/treatment/prescription. 4. Validate. 5. Save record.	Unauthorized access → deny operation.
UC-04	Admit Patient	Receptionist/Admin	Patient exists and room/bed is available	1. Select admission. 2. Enter patient and doctor. 3. View available room/bed. 4. Assign room. 5. Save admission.	No available bed → display availability message.
UC-05	Generate Bill	Billing Staff	Patient has billable services	1. Select Billing. 2. Enter patient ID. 3. Add charges. 4. Calculate total. 5. Save bill. 6. Record payment if applicable.	Invalid amount → reject and request correction.
UC-06	Search Patient	Authorized User	User is logged in	1. Select Search. 2. Enter ID/name. 3. Retrieve matching records. 4. Display permitted information.	No match → display 'No record found'.

10. Testing and Acceptance Criteria

10.1 Testing Strategy

- Unit testing: test individual functions such as validation, ID generation, calculations, and file operations.
- Integration testing: verify interactions between patient, appointment, medical, admission, and billing modules.
- System testing: execute complete user workflows from login to report generation.
- Validation testing: enter invalid, empty, duplicate, and boundary-value inputs.
- User acceptance testing: verify that the implemented system satisfies the SRS requirements.

10.2 Representative Test Cases

Test ID	Scenario	Expected Result
TC-01	Register patient with valid data	Patient record is saved with a unique Patient ID.

TC-02	Register patient with missing mandatory field	System rejects input and identifies the missing field.
TC-03	Search for existing patient by ID	Correct patient record is displayed.
TC-04	Book appointment for valid patient and doctor	Appointment is created with a unique Appointment ID.
TC-05	Book appointment using invalid Patient ID	Operation is rejected.
TC-06	Add medical record	Record is saved and linked to patient and doctor.
TC-07	Admit patient to unavailable room	System prevents allocation and displays an error.
TC-08	Create bill and record payment	Total and outstanding balance are calculated correctly.
TC-09	Restart application after saving data	Previously saved records remain available.
TC-10	Exit application normally	Application closes without losing committed data.

10.3 Acceptance Criteria

- All high-priority functional requirements are implemented and demonstrable.
- Core patient, doctor, appointment, medical record, admission, and billing workflows operate correctly.
- Invalid inputs are handled without unexpected application termination.
- Persistent records remain available after restarting the application.
- The system can generate at least the basic reports specified in the requirements.
- The source code compiles successfully in the agreed C/C++ development environment.

11. Future Enhancements

- Database integration using MySQL, PostgreSQL, SQLite, or another DBMS.
- Graphical user interface and web/mobile front end.
- Online appointment booking and patient self-service.
- SMS/email reminders for appointments.
- Laboratory and pharmacy modules.
- Insurance claim processing.
- Role-based audit logs and stronger authentication.
- Cloud deployment and multi-user network access.
- Dashboard with hospital statistics and analytics.
- Integration with external healthcare interoperability standards where appropriate.

12. Conclusion

The Hospital Management System provides a structured solution for managing essential hospital operations in an academic mini-project environment. The SRS defines the system scope, users, functional and non-functional requirements, data entities, interfaces, use cases, constraints, and acceptance criteria. The proposed modular C/C++ implementation is intentionally focused on core hospital workflows while leaving a clear path for future database, GUI, networking, and integration enhancements.

Appendix A – Data Dictionary

Field	Type (Suggested)	Description
PatientID	String/Integer	Unique patient identifier
PatientName	String	Full name of patient
DOB/Age	Date/Integer	Date of birth or age
Gender	String/Enum	Patient gender
Contact	String	Telephone/mobile contact
DoctorID	String/Integer	Unique doctor identifier
Specialization	String	Doctor's medical specialization
AppointmentID	String/Integer	Unique appointment identifier
AppointmentDate	Date	Date of appointment
AppointmentTime	Time	Scheduled time
AdmissionID	String/Integer	Unique admission identifier
RoomID	String/Integer	Unique room/bed identifier
RecordID	String/Integer	Unique medical record identifier
Diagnosis	String	Recorded diagnosis
Treatment	String	Treatment or clinical notes
BillID	String/Integer	Unique billing identifier
TotalAmount	Decimal	Total amount payable
PaidAmount	Decimal	Amount already paid
Balance	Decimal	Outstanding amount

Appendix B – Suggested C/C++ Program Structure

- main.cpp – application entry point and main menu.
- patient.cpp / patient.h – patient data and operations.
- doctor.cpp / doctor.h – doctor management.
- appointment.cpp / appointment.h – appointment scheduling.
- admission.cpp / admission.h – admission and room management.
- medical_record.cpp / medical_record.h – diagnosis and treatment records.
- billing.cpp / billing.h – billing and payments.
- reports.cpp / reports.h – report generation.
- storage.cpp / storage.h – file handling and persistence.
- validation.cpp / validation.h – input validation and helper functions.